

AsteroidsFX

A Component-Oriented Game on the Java Platform Module System

Designing, implementing and validating runtime-replaceable components

Student name	[INSERT YOUR FULL NAME]
Student exam number	[INSERT YOUR EXAM NUMBER]
GitHub user	Fard121
GitHub repository	https://github.com/Fard121/asteroid_game
Forked from	https://github.com/sweat-tek/AsteroidsFX
Video demonstration	[INSERT YOUTUBE LINK]
Course	Component Based Systems, SDU MMMI
Contact	fjama23@student.sdu.dk
Date	9 August 2026

Fields shown in red must be completed before submission.

Table of Contents

Abstract.....	4
1 Introduction.....	5
1.1 Background and motivation.....	5
1.2 Aim and scope.....	5
1.3 Structure of this report.....	6
2 Requirements.....	7
2.1 Functional requirements.....	7
2.2 Non-functional requirements.....	8
2.3 Mandated components and their interfaces.....	8
3 Analysis.....	10
3.1 Use cases.....	10
3.2 Domain model.....	11
3.3 Behavioural model.....	12
3.4 Identified and missing components.....	12
4 Design.....	14
4.1 Architectural overview.....	14
4.2 Components and connections.....	15
4.3 Operation contracts.....	16
4.4 The two-phase frame.....	17
4.5 Module layers and split packages.....	17
4.6 The scoring boundary.....	18
5 Implementation.....	20
5.1 Registration and access of components.....	20
5.2 Reliable configuration and strong encapsulation.....	21
5.3 Component models applied.....	22
5.4 Dynamic installation and uninstallation.....	23
5.5 Collision, splitting and scoring.....	24
5.6 The scoring client.....	25
6 Test.....	27
6.1 Unit testing strategy.....	27
6.2 Build and unit test results.....	28
6.3 Integration in a real deployment.....	30
6.4 Experimental validation of dynamic update.....	32
6.5 Summary of results.....	35
7 Discussion.....	37
7.1 How well the design met the requirements.....	37

7.2 Trade-offs accepted.....	37
7.3 Limitations.....	38
7.4 Reflection on the development process.....	38
8 Conclusion.....	39
9 References.....	39
Appendix A Repository map.....	42
Appendix B Reproducing the results.....	42

Abstract

This report documents the design, implementation and validation of AsteroidsFX, a component-oriented reimplement of the arcade game Asteroids built on the Java Platform Module System (JPMS), JavaFX and the Spring container. The engineering problem addressed is not the game itself but the coupling between the parts that make it. In a conventional implementation the main loop must name every gameplay subsystem it drives, so no subsystem can be added, removed or replaced without recompiling and relinking that loop. The solution presented here inverts that relationship. Five gameplay components, Player, Enemy, Bullet, Asteroids and Collision, are expressed purely as providers of service interfaces declared in shared API modules, are discovered at runtime by ServiceLoader from a plugins directory, and are resolved into a dedicated child ModuleLayer. The composition root, Core, declares no requires clause for any of them, and the Spring container injects whatever the module layer yields. Score keeping is externalised further still, into a separate Spring Boot process reached over HTTP. The result is validated on three levels: 21 JUnit 5 and Mockito unit tests covering the component contracts, an integration run of the complete two-process deployment, and an experimental demonstration that a component can be uninstalled and reinstalled both at runtime and at deployment time without recompiling any source file.

Keywords: *component-based software engineering, Java Platform Module System, ServiceLoader, ModuleLayer, whiteboard component model, dependency injection, microservices, operation contracts, JavaFX.*

1 Introduction

1.1 Background and motivation

Component-based software engineering treats a system as an assembly of independently deployable units that interact only through explicitly declared interfaces [5]. The promise is substitution: if a unit's obligations are fully captured by its contract, any implementation honouring that contract can replace it without disturbing the rest of the system. The difficulty is that ordinary language mechanisms work against this, because a direct import creates a compile-time dependency, and a compile-time dependency is precisely what prevents substitution at deployment time.

Java addressed this in two stages. `ServiceLoader` decouples a consumer from a provider by resolving implementations at runtime rather than at compile time [3], [4]. The Java Platform Module System, delivered in Java 9 under JSR 376, added the missing half: reliable configuration and strong encapsulation, so that the dependency graph is validated when the application starts and a module's internals are genuinely unreachable unless deliberately exported [1], [2]. Together they allow a system to be assembled from parts that it never names.

1.2 Aim and scope

The aim of this project was to build a working Asteroids game whose gameplay subsystems are true components in the sense above, and then to demonstrate empirically that one of them can be removed and restored without recompiling the source code. The scope covers the nine laboratory exercises of the course [12]: the initial JavaFX game, the component and data-oriented refactoring, `ServiceLoader` assembly, the three JPMS exercises on module descriptors, services and module layers, Spring dependency injection in the composition root, the scoring microservice, and unit testing of the components.

Persistence of high scores, networked multiplayer, and hot reloading of components that were not present when the process started are outside scope. Section 7 explains why the last of these is a deliberate boundary rather than an omission.

1.3 Structure of this report

Section 2 states the functional and non-functional requirements and the interface contracts they imply. Section 3 analyses the problem as use cases, a domain model and a behavioural model, without committing to a solution. Section 4 presents the architecture and the operation contracts of every provided and required interface. Section 5 explains how components are registered, discovered and accessed in the source code, and where each component model is applied. Section 6 reports experimental validation, including the dynamic update experiment. Sections 7 and 8 discuss how well the design met its requirements and conclude.

2 Requirements

Requirements are separated into functional requirements, which describe observable behaviour, and non-functional requirements, which constrain how that behaviour is achieved. For this project the non-functional requirements carry most of the engineering weight: a monolithic implementation could satisfy every functional requirement below and still fail the exercise entirely.

2.1 Functional requirements

ID	Requirement	Component responsible
FR1	The player controls a ship that rotates, thrusts with inertia and wraps around the screen edges	Player
FR2	The player fires bullets, subject to a cooldown and a bounded number in flight	Player, Bullet
FR3	Asteroids drift with random headings and wrap around the screen edges	Asteroids
FR4	An enemy saucer moves randomly and fires at intervals	Enemy
FR5	Collision between entities is detected using Pythagorean distance	Collision
FR6	An asteroid hit by a bullet splits into two smaller ones; the smallest is destroyed outright	Collision, Asteroids
FR7	Ships that collide with an asteroid are destroyed	Collision
FR8	The player ship and the enemy saucer are destroyed after a number of bullet hits	Collision
FR9	The score increases by an amount determined by the target destroyed	Collision
FR10	Waves advance when the field is cleared, raising count and speed; wave 3 is victory	Asteroids
FR11	The game presents start and pause menus, a help overlay, and game over and victory states	Core
FR12	The current score is published to an external scoring service	Core, Scoring

Table 1. Functional requirements and the component responsible for each.

2.2 Non-functional requirements

ID	Requirement	Rationale
NFR1	A gameplay component must be installable and uninstallable without recompiling any source file	The central objective of the course; verified experimentally in Section 6.4
NFR2	The composition root must not name any gameplay component at compile time	A requires clause would reintroduce exactly the coupling NFR1 removes
NFR3	Component internals must be unreachable from other components	Strong encapsulation: only the declared interfaces form the contract [1]
NFR4	The dependency configuration must be validated at start-up, not when a class is first touched	Reliable configuration: a missing module should fail loudly and early [2]
NFR5	Every component must be unit-testable in isolation, without a running JavaFX stage	A component that can only be exercised by running the whole system is not a component
NFR6	Failure of the external scoring service must not stop the game loop	Availability across a process boundary; see Section 4.6

Table 2. Non-functional requirements.

2.3 Mandated components and their interfaces

The exercise requires five components in particular: Player, Enemy, Asteroids, Weapon, realised here as the Bullet component, and the rendering and drawing component, realised as Core. Player, Enemy and Weapon must implement provided service interfaces so that they can be updated or removed without recompilation. Table 3 states the resulting contract obligations.

Component	Provides	Requires (uses)	Deployed in
Core	Composition root, rendering, HUD and input; provides no service	The three processing interfaces, indirectly via Common	mods-mvn/
Player	IGamePluginService`, IEntityProcessingService	BulletSPI	plugins/
Enemy	IGamePluginService`, IEntityProcessingService	BulletSPI	plugins/
Bullet (Weapon)	IGamePluginService`, IEntityProcessingService`, `BulletSPI	none	plugins/
Asteroids	IGamePluginService`, IEntityProcessingService`, `IAsteroidSplitter	none	plugins/
Collision	IPostEntityProcessingService	IAsteroidSplitter	plugins/
Scoring	REST API on port 8081	none	separate process

Table 3. Mandated components with their provided and required interfaces.

3 Analysis

This section describes what the system must do and what it must contain. It deliberately avoids stating how any of it is achieved, which is the subject of Sections 4 and 5.

3.1 Use cases

Two actors interact with the system. The player drives gameplay. A second actor, the integrator, represents whoever assembles or reconfigures the deployment. This actor is unusual in a game, and its presence is a direct consequence of NFR1: use case UC9 exists only because components are meant to be replaceable, and it is the use case the video demonstration exercises.

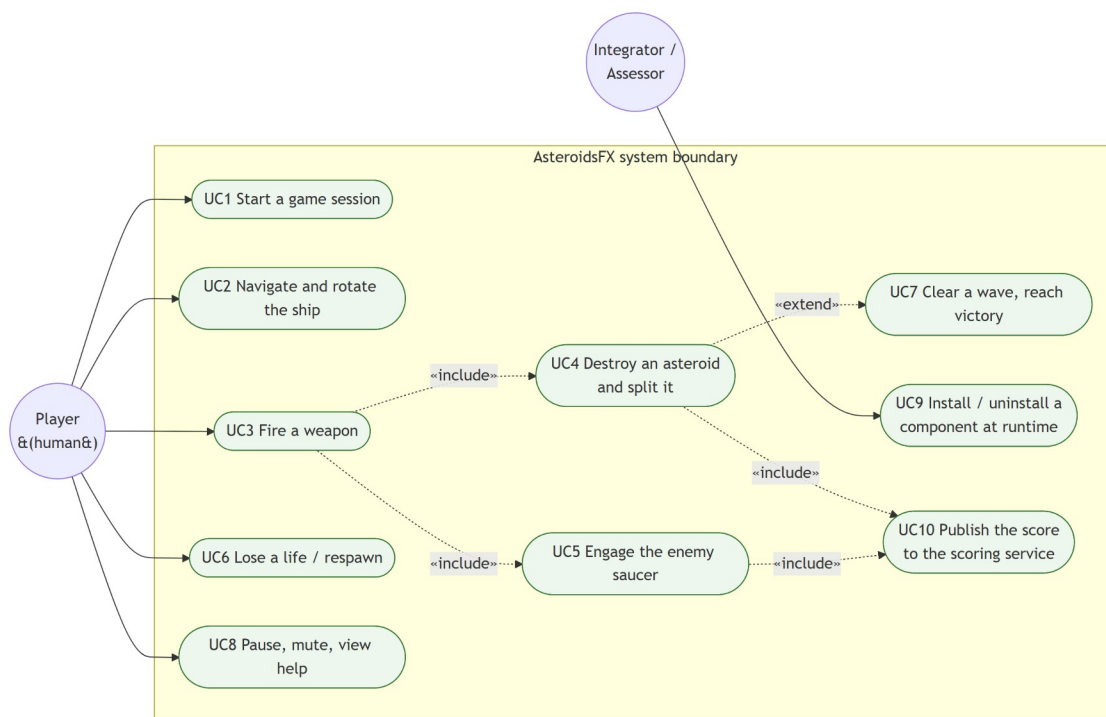


Figure 1. Use case model. The integrator actor and UC9 exist because of the component-replaceability requirement rather than because of gameplay.

Field	UC9 Install or uninstall a component at runtime
Primary actor	Integrator
Precondition	The application is running and the target component was discovered at start-up
Trigger	The integrator presses key 1, 2 or 3 for the Player, Enemy or Weapon component
Main flow	1. Core resolves the key to a named component. 2. If the component is installed, its IGamePluginService.stop is invoked and its services are removed from the active

	lists. 3. If it is uninstalled, start is invoked and its services are added back.
Postcondition	The component's entities and per-frame behaviour are absent from, or present in, the running game, and no other component is affected
Guarantee	No source file is recompiled and the JVM is not restarted

Table 4. Expanded description of the use case central to the exercise.

3.2 Domain model

The game state is data-oriented [14]: entities are plain data carriers and behaviour lives in separate processing services, rather than each entity owning its own update method. That separation is what makes behaviour relocatable into a component at all. Entity holds position, rotation, a collision radius, a polygon outline, a category and hit points. World is a keyed collection of entities, and GameData aggregates the session state that every component may read.

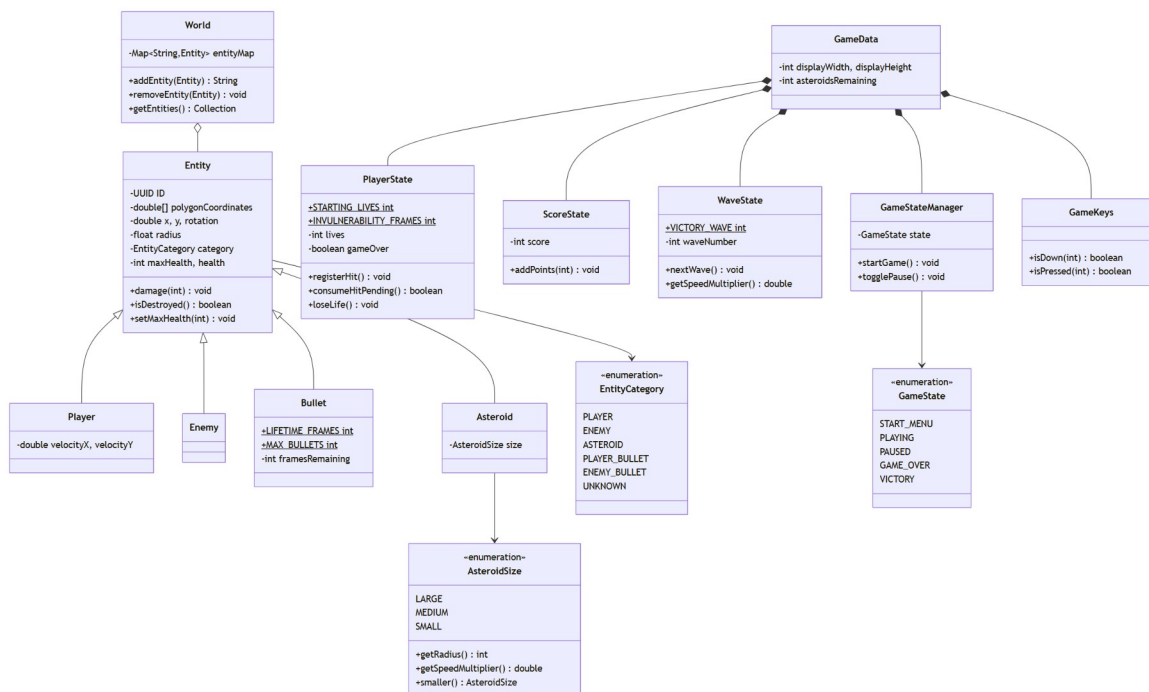


Figure 2. Domain model. Bullet and Asteroid are declared in shared API modules because several components must recognise them; Player and Enemy are declared inside their own components and are named by no one else.

The distinction visible in the diagram matters. Bullet and Asteroid live in CommonBullet and CommonAsteroids because more than one component must recognise them: Player and Enemy count bullets in flight, and Collision asks an asteroid for its size. Player and Enemy, by contrast, are declared inside their own components, and Collision distinguishes a player from an asteroid

using the EntityCategory enumeration rather than by type. That is why the Collision component compiles with no knowledge whatsoever of the Player component.

3.3 Behavioural model

Gameplay is a state machine owned by GameStateManager. Only in the playing and terminal states are the entity processors executed. In the menu and paused states the world is rendered but not advanced, which is what allows a menu to be drawn over a frozen field.

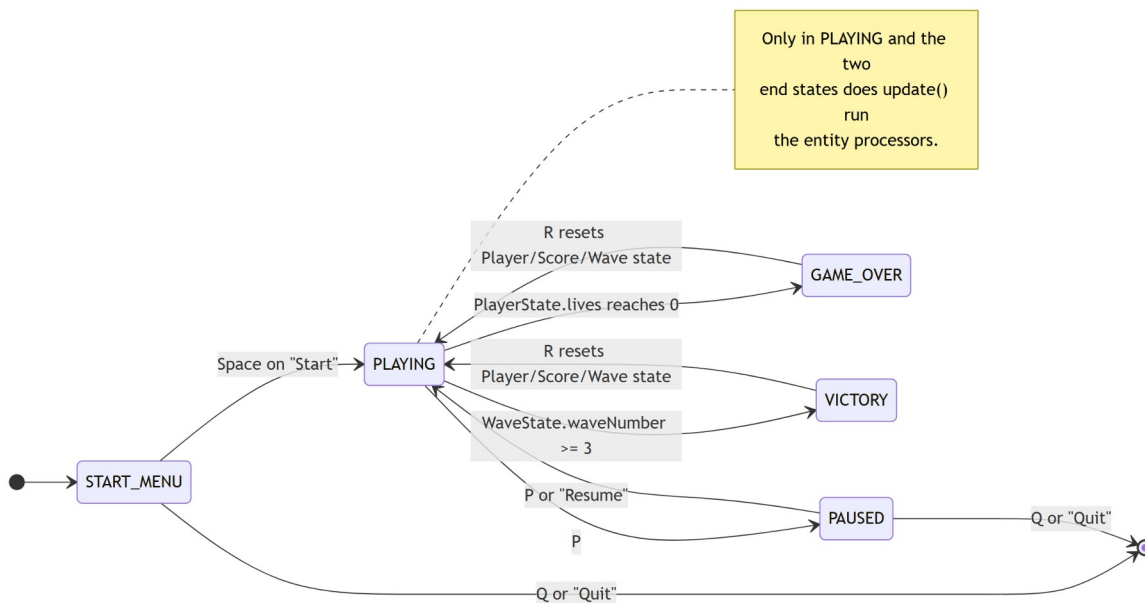


Figure 3. Game state machine as implemented by GameStateManager and interpreted each frame by Core.

3.4 Identified and missing components

The analysis identifies six in-process components, Player, Enemy, Bullet, Asteroids, Collision and Core, and one out-of-process component, Scoring. It also identifies capabilities that exist in the behaviour but are not owned by a component of their own:

- Spawning and difficulty. Wave progression lives inside AsteroidProcessor and enemy spawn timing inside EnemyControlSystem. A dedicated spawn service would allow difficulty to be tuned without modifying either component.
- Score persistence. The scoring service holds the score in memory only, so no component is responsible for retaining a high score between runs.

- Audio. SoundManager is a static utility in the shared module rather than a replaceable provider, so an alternative sound implementation cannot be substituted the way a gameplay component can.

These are recorded rather than resolved. Each would be a straightforward addition using the service mechanism already in place, which is itself evidence that the architecture generalises beyond the components it currently carries.

4 Design

4.1 Architectural overview

The system is deployed as two operating system processes. The game process contains two module layers: the boot layer, holding the composition root and the shared API modules, and a child layer holding the five gameplay components. The scoring process is a self-contained Spring Boot application reached over HTTP. The two-layer split inside a single process is the architectural decision on which every other one depends.

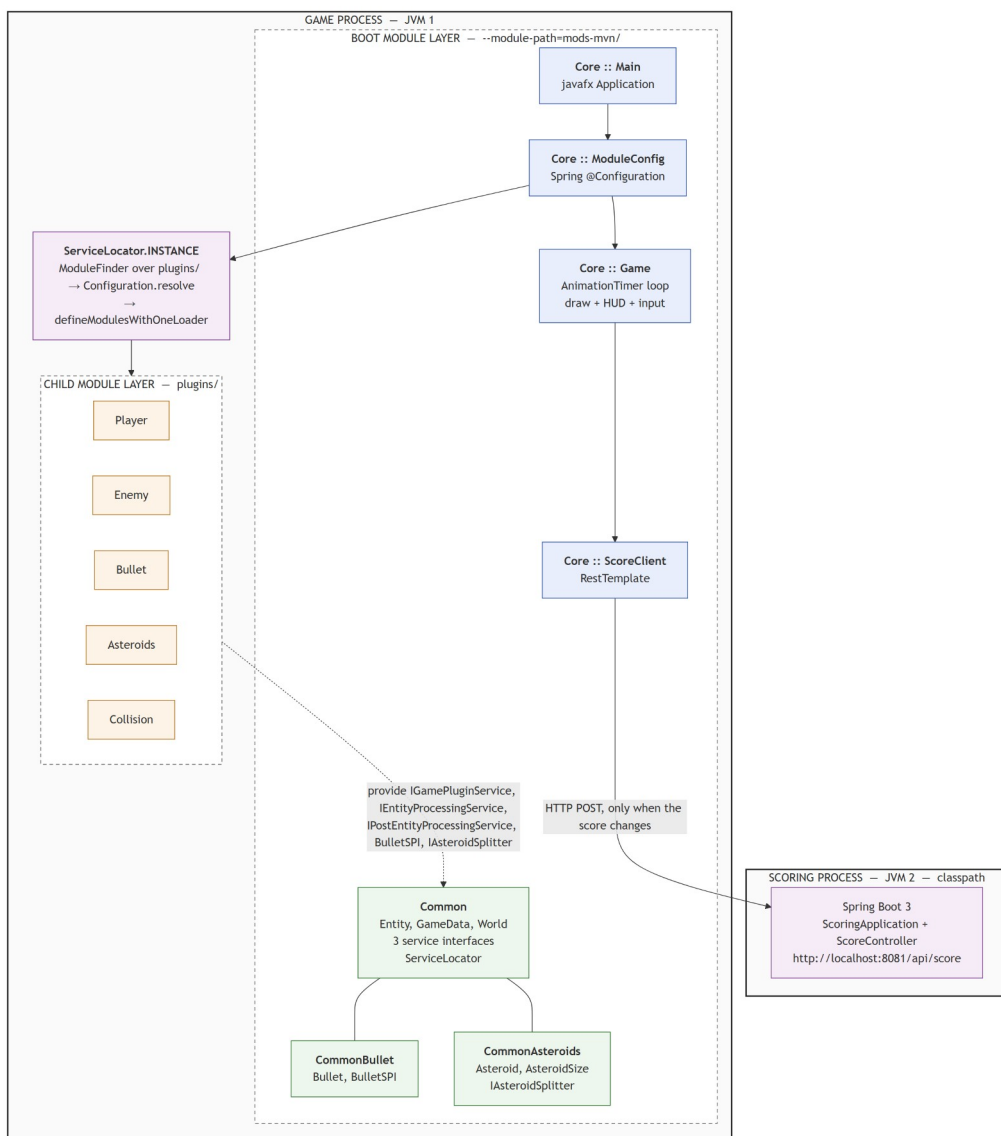


Figure 4. System architecture. Core sits in the boot layer and knows only the interfaces in Common. The five gameplay components are discovered in plugins/ and resolved into a child ModuleLayer at start-up. The score crosses a process boundary over HTTP.

Four architectural principles govern the structure:

- Composition root. Core owns the game loop but declares no requires clause for any gameplay component, satisfying NFR2.
- Interface segregation. Contracts live in Common, CommonBullet and CommonAsteroids, so a component depends on a contract rather than on a peer.
- Layer isolation. Component jars are resolved into their own configuration and class loader, so their resolution cannot corrupt the boot layer's module graph.
- Externalised state. The authoritative score lives in a separate process, which forces the interaction to be an explicit and failure-tolerant protocol rather than a method call.

4.2 Components and connections

Figure 5 shows the assembly in UML component terms. Every connection is a service contract: a provided interface on one side, a required interface on the other. No connection in the diagram represents a compile-time dependency between two gameplay components.

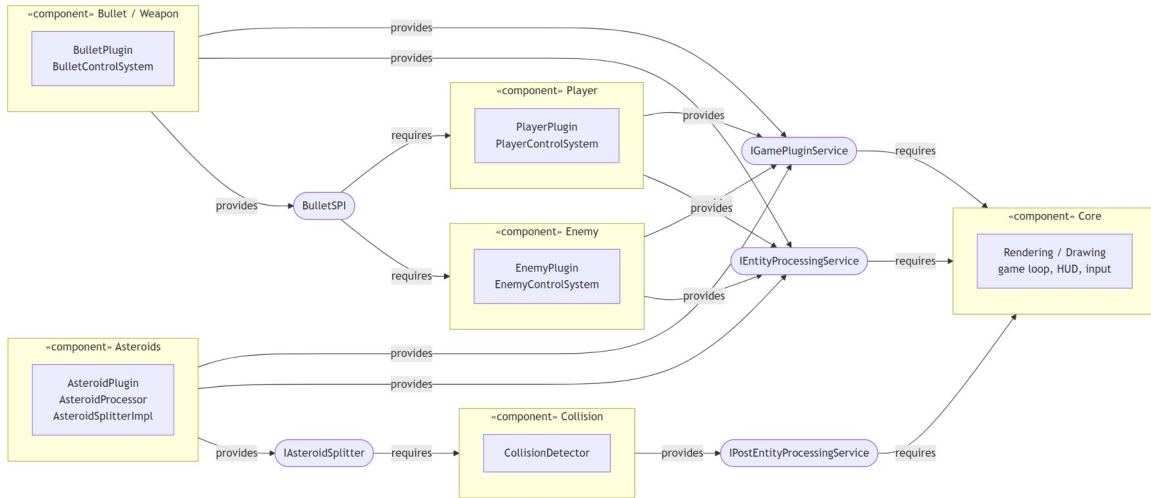


Figure 5. UML component diagram. Player and Enemy require BulletSPI, which Bullet provides; Collision requires IAsteroidSplitter, which Asteroids provides. Neither pair shares a compile-time dependency.

Figure 6 shows the same system from the module system's point of view. Solid edges are requires clauses resolved when the application starts; dashed edges are uses relationships resolved at

runtime by ServiceLoader. The absence of any solid edge from Core to a gameplay module is the structural proof of NFR2.

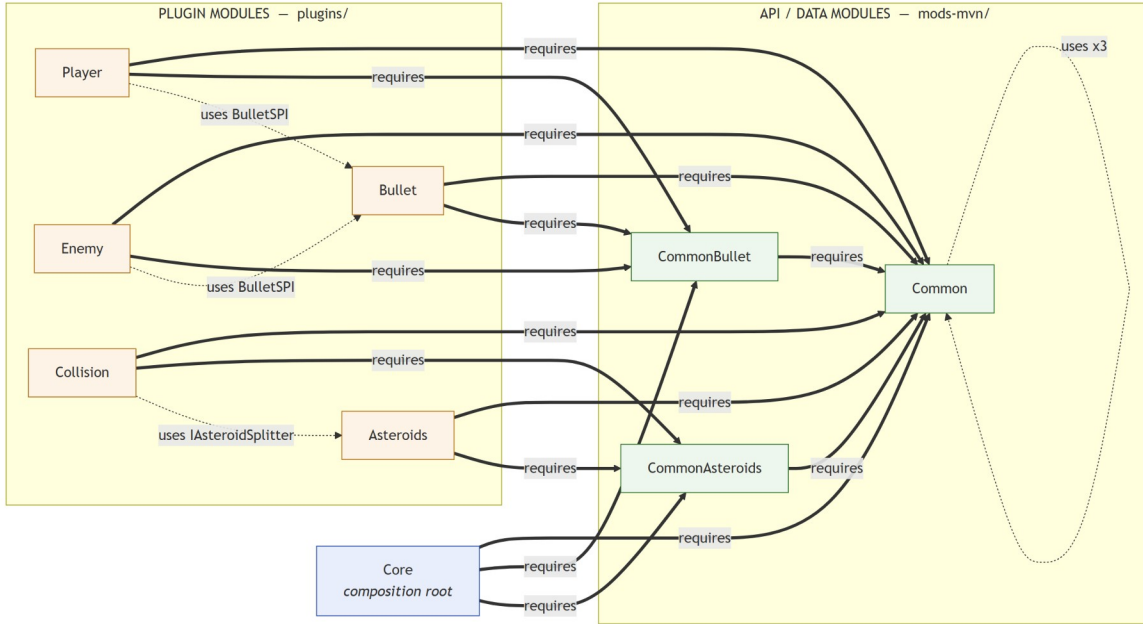


Figure 6. Module dependency graph. Solid edges are *requires* clauses, dashed edges are runtime service lookups. Core has no solid edge to any gameplay module.

4.3 Operation contracts

Each interface operation is specified as a contract in the style of the Unified Process [13]: a precondition the caller must establish and a postcondition the implementation guarantees. These contracts are written as JavaDoc on the interfaces themselves, so they travel with the code rather than only with this document.

Operation	Precondition	Postcondition
<code>IGamePluginService.start</code>	gameData and world are non-null and represent a fresh session; the world may already hold entities added by plugins started earlier, but none of this plugin's own	Entities this plugin owns up front have been added to the world, and plugin-local state needed by its processor has been initialised
<code>IGamePluginService.stop</code>	start was previously called on this instance with the same gameData and world	Every entity exclusively owned by this plugin has been removed and its resources released; calling stop without a prior start is a no-op, not an error
<code>IEntityProcessingService.process</code>	Called once per frame in no guaranteed order relative to other implementations, so it may not assume any peer has already run this frame	Every entity this component owns has advanced exactly one frame and any entities it spawns have been added; it never removes another component's entities
<code>IPostEntityProcessingService.process</code>	Every <code>IEntityProcessingService</code> for this frame has completed, so all entity	Cross-entity effects are fully applied for the frame; the implementation must

	positions are final	snapshot the world before mutating it so that no pair is skipped or processed twice
BulletSPI.createBullet	The shooter is non-null and has a valid position and rotation	A fully initialised bullet is returned but not added to the world, or empty if the weapon component is not currently installed; adding it is the caller's responsibility
IAsteroidSplitter.createSplitAsteroid	The entity is an asteroid that the caller has removed or is about to remove; this operation does not remove it	Two smaller asteroids have been added at the destroyed asteroid's position, or none if it was already the smallest size

Table 5. Operation contracts for every provided and required interface.

4.4 The two-phase frame

The precondition on `IEntityProcessingService.process`, that implementations run in no guaranteed order, is the reason collision detection cannot be one of them. If collision ran as an ordinary processor it could observe some entities at their new positions and others at their old ones, and the outcome would depend on the order in which `ServiceLoader` happened to return providers. Dividing the frame into an entity phase and a post-entity phase makes the ordering guarantee explicit in the type system: any logic that requires settled positions must implement the second interface, and the compiler, rather than a comment, enforces the distinction.

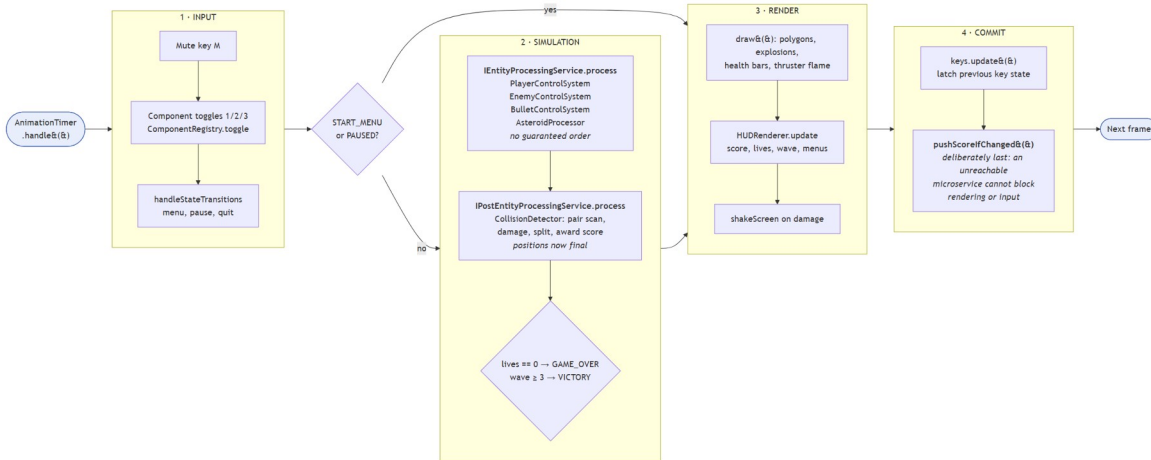


Figure 7. Per-frame control flow. The simulation phases are skipped in the menu and paused states, and score synchronisation deliberately runs last.

4.5 Module layers and split packages

Two modules that export the same package cannot be defined under one class loader; the module system rejects the attempt with a `LayerInstantiationException`. This was reproduced deliberately in a minimal two-module demonstration kept under `docs/jpms-lab3-demo`, and the captured

behaviour is instructive. `Configuration.resolve` alone does not detect the clash, because it resolves only the requires graph. The failure appears when the modules are actually defined together, which is exactly what happens when everything sits on one flat module path.

The resolution is to give each conflicting module its own layer, and therefore its own loader. AsteroidsFX applies the same mechanism for a related reason: because the plugin set is resolved as its own isolated configuration, a package clash between two components fails loudly and specifically when `ServiceLocator` is constructed, instead of silently corrupting the boot layer's module graph.

4.6 The scoring boundary

Moving the score out of process turns a field access into a network call, and a network call can fail. NFR6 requires that this failure be contained, and two design decisions achieve it. First, synchronisation runs as the last statement of the frame, after rendering and input handling have already completed, so an exception cannot abort the remainder of the frame. Second, the client publishes only on an actual change of score, and after a failure backs off for roughly three seconds rather than retrying every frame.

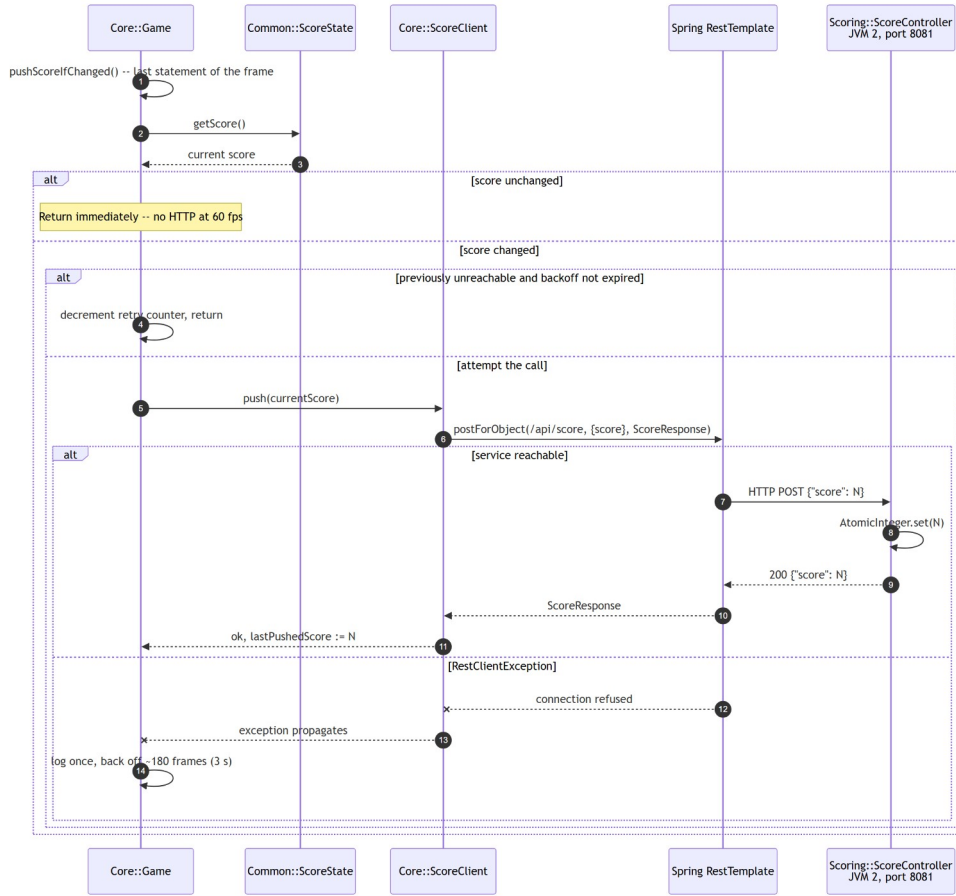


Figure 8. Score synchronisation including the failure path. The game continues at full frame rate whether or not the scoring service is reachable.

This design was not arrived at analytically. An earlier version placed the call before rendering, and when the scoring service was stopped the uncaught exception aborted the rest of the frame: the screen stopped redrawing and input stopped responding, which presented exactly as a frozen game. The ordering is therefore a corrective response to an observed defect, and is documented as such in the repository.

5 Implementation

This section explains how the design is realised in source code: how components are registered and accessed, how reliable configuration and strong encapsulation are obtained, and where each component model appears in the repository.

5.1 Registration and access of components

Components are never registered explicitly. A component joins the running game solely by declaring a provides clause in its module descriptor; nothing imports it and nothing lists it. The single point of assembly is ServiceLocator in the Common module, which builds the child module layer once and then answers lookups against it.

```
Path pluginsDir = Paths.get("plugins");
ModuleFinder pluginsFinder = ModuleFinder.of(pluginsDir);

List<String> plugins = pluginsFinder.findAll().stream()
    .map(ModuleReference::descriptor)
    .map(ModuleDescriptor::name)
    .collect(Collectors.toList());

Configuration pluginsConfiguration = ModuleLayer
    .boot().configuration()
    .resolve(pluginsFinder, ModuleFinder.of(), plugins);

layer = ModuleLayer.boot()
    .defineModulesWithOneLoader(pluginsConfiguration,
        ClassLoader.getSystemClassLoader());
```

*Listing 6. ServiceLocator builds the child ModuleLayer from whatever jars are present in plugins/
Common/src/main/java/dk/sdu/mmmi/cbse/common/util/ServiceLocator.java*

The set of components is therefore determined by the contents of a directory at start-up, not by anything written in the source. Figure 9 traces the full bootstrap, from the launch command to the first frame.

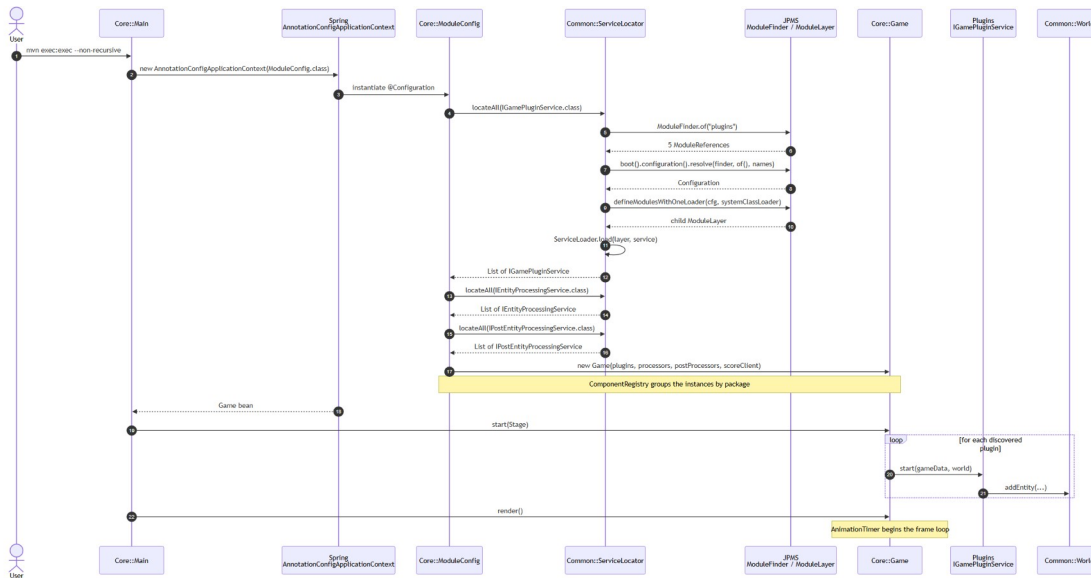


Figure 9. Application start-up. Spring asks ServiceLocator for each service type, ServiceLocator resolves the plugins directory into a child ModuleLayer, and Game receives the discovered lists without ever naming a concrete component.

5.2 Reliable configuration and strong encapsulation

The module descriptors are where NFR2, NFR3 and NFR4 are enforced. Two properties are worth reading carefully in the excerpt below. Core requires only shared API modules, so the compiler physically cannot allow it to reach into a gameplay component. The Player component declares uses BulletSPI but never requires Bullet, so it compiles, and runs, whether or not a weapon implementation exists.

```

module Core {
  requires Common;
  requires CommonBullet;
  requires CommonAsteroids;
  dk.sdu.mmmi.cbse.common.bullet.BulletSPI;
  requires javafx.graphics;
  requires spring.context;
  exports dk.sdu.mmmi.cbse.main;
  opens dk.sdu.mmmi.cbse.main
    to javafx.graphics, spring.core;
  // no requires Player / Enemy /
  // Bullet / Asteroids / Collision
}

module Player {
  requires Common;
  requires CommonBullet;
  uses
    provides IGamePluginService
      with ...playersystem.PlayerPlugin;
    provides IEntityProcessingService
      with ...playersystem.PlayerControlSystem;
}

module Bullet {
  requires Common;
  requires CommonBullet;
  provides BulletSPI
    with ...bulletsystem.BulletControlSystem;
}

```

Listing 7. Module descriptors. Core names no gameplay module; Player consumes BulletSPI without depending on the component that provides it.

Encapsulation is asymmetric by design. The shared modules export their packages because their types are the contract. The five gameplay components export nothing at all: their classes are reachable only as service implementations, so no other component can bind to their internals even accidentally. This is a stricter reading of the exercise than exporting every package would be, and it is what makes substitution safe.

5.3 Component models applied

Three distinct component models are used, each in the place where it fits. Their coexistence is deliberate: JPMS supplies reliable configuration, ServiceLoader supplies late binding, and Spring supplies runtime wiring.

Component model	Where it is applied	Source location
Whiteboard model [15]	A component publishes an implementation and is called by the framework; it never registers itself with, or looks up, the framework	Common/.../services/*.java` and every plugin module-info
JPMS services	provides ... with and uses declarations express the contract at module granularity, checked when the configuration resolves	*/src/main/java/module-info.java
ServiceLoader with ModuleLayer	Late binding of implementations discovered from a directory rather than a class path	Common/.../util/ServiceLocator.java
Dependency injection	The discovered lists are turned into beans and injected into the Game constructor	Core/.../main/ModuleConfig.java
Microservice [17], [18]	The authoritative score is owned by a separate process behind a REST interface	Scoring/.../ScoreController.java`, `Core/.../ScoreClient.java

Table 8. Component models applied and where each appears in the repository.

```
@Configuration
class ModuleConfig {

    @Bean public Game game() {
        return new Game(gamePluginServices(), entityProcessingServiceList(),
            postEntityProcessingServices(), scoreClient());
    }

    @Bean public List<IEntityProcessingService> entityProcessingServiceList() {
        return ServiceLocator.INSTANCE.locateAll(IEntityProcessingService.class);
    }
    ...
}
```

Listing 9. The Spring configuration wires whatever the module layer yielded.
Core/src/main/java/dk/sdu/mmmi/cbse/main/ModuleConfig.java

5.4 Dynamic installation and uninstallation

ComponentRegistry, held by Core, groups the already discovered service instances by the package of their implementing class and maintains the lists that the frame loop iterates. Installing or uninstalling a component is therefore a list operation plus a lifecycle call; no discovery is repeated, no class is reloaded, and nothing is recompiled.

```
void toggle(String name, GameData gameData, World world) {
    for (Component component : components) {
        if (!component.name.equals(name)) continue;
        if (component.installed) {
            for (IGamePluginService plugin : component.plugins) plugin.stop(gameData,
world);
            activePlugins.removeAll(component.plugins);
            activeProcessors.removeAll(component.processors);
        } else {
            for (IGamePluginService plugin : component.plugins) plugin.start(gameData,
world);
            activePlugins.addAll(component.plugins);
            activeProcessors.addAll(component.processors);
        }
        component.installed = !component.installed;
        return;
    }
}
```

*Listing 10. Runtime installation and uninstallation.
Core/src/main/java/dk/sdu/mmmi/cbse/main/ComponentRegistry.java*

The weapon component illustrates why the contract must anticipate absence. BulletSPI.createBullet returns an Optional, and when the component is uninstalled it returns empty. Player and Enemy therefore keep running normally and simply stop producing bullets; neither contains any knowledge that the weapon can disappear beyond honouring the Optional. Figure 11 traces the sequence.

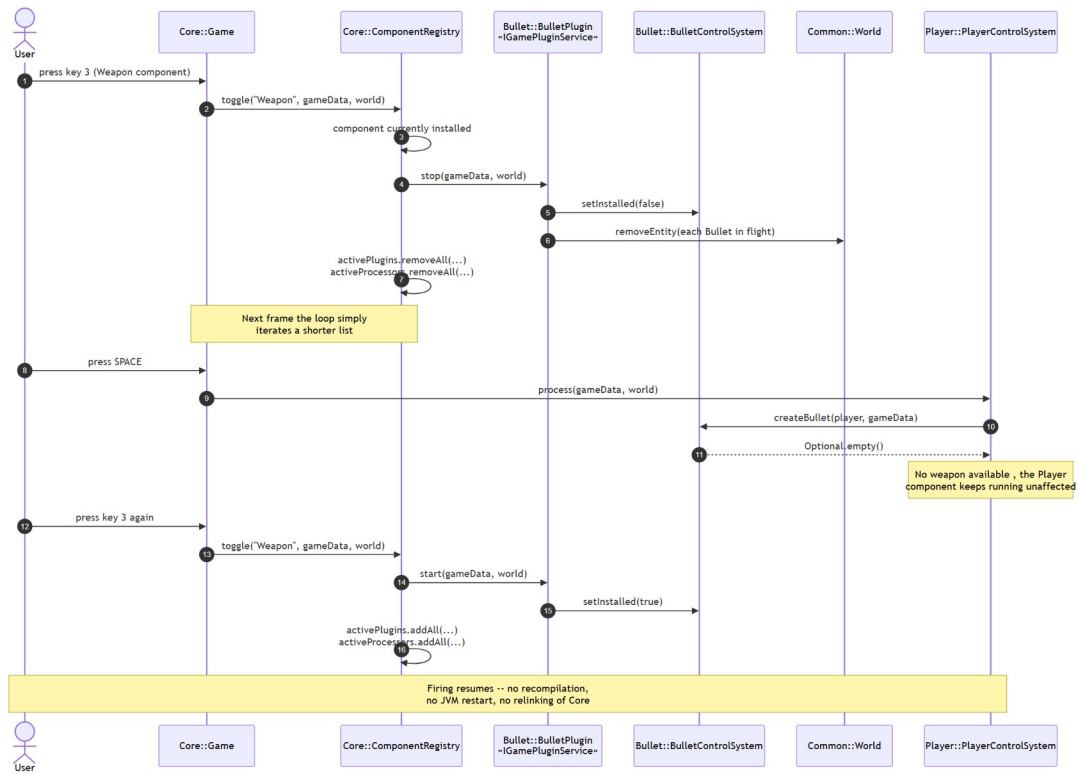


Figure 10. Runtime uninstallation and reinstallation of the weapon component. The Player component continues to run throughout, receiving an empty Optional while no weapon is installed.

5.5 Collision, splitting and scoring

CollisionDetector is the only implementer of IPostEntityProcessingService. It filters candidate pairs by category before performing any distance arithmetic, so friendly fire and asteroid-against-asteroid contacts are never considered, and it defers all removals until the scan is complete so that no pair is skipped or processed twice.

```

public Boolean collides(Entity entity1, Entity entity2) {
    float dx = (float) entity1.getX() - (float) entity2.getX();
    float dy = (float) entity1.getY() - (float) entity2.getY();
    float distance = (float) Math.sqrt(dx * dx + dy * dy);
    return distance < (entity1.getRadius() + entity2.getRadius());
}

```

Listing 11. Pythagorean collision test.
Collision/src/main/java/dk/sdu/mmmi/cbse/collisionsystem/CollisionDetector.java

Three destruction paths are distinguished, and the difference between them is a design decision rather than an accident. A collision involving the player never removes the player entity; it registers a hit that the Player component consumes on the following frame, which is what allows lives and respawn invulnerability to live entirely inside that component. A ship that flies into an

asteroid is destroyed outright, because a rock is not a bullet. Only a bullet kill awards points, so neither ramming nor a ship wrecked by an asteroid inflates the score.

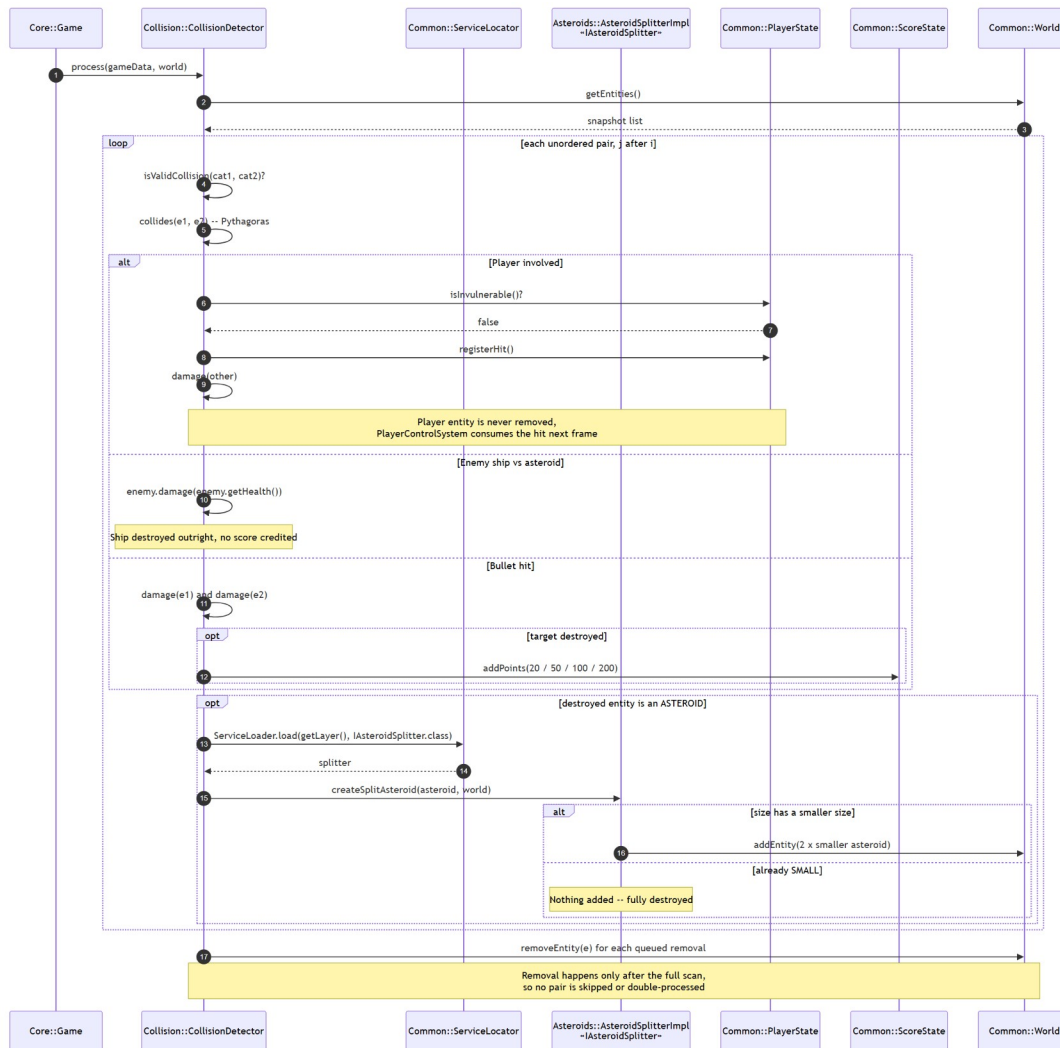


Figure 11. Collision resolution. The splitter is reached through *IAsteroidSplitter*, so the Collision component never names a concrete asteroid type.

Note in the figure that Collision obtains the splitter through a service lookup. It knows that something can split an asteroid; it does not know what, and if the Asteroids component were absent the lookup would simply yield nothing.

5.6 The scoring client

ScoreClient is a thin wrapper over Spring's RestTemplate [6]. Making a plain JPMS module use RestTemplate required three additions to the module descriptor that are worth recording, because

each was discovered by running the application rather than by reading documentation: `spring.web` itself; `micrometer.observation`, which `RestTemplate` references during class initialisation; and `com.fasterxml.jackson.databind`, without which no JSON message converter is registered and posting an object fails outright.

```
class ScoreClient {
    private static final String SCORE_URL = "http://localhost:8081/api/score";
    private final RestTemplate restTemplate = new RestTemplate();

    void push(int score) {
        restTemplate.postForObject(SCORE_URL, new ScoreUpdateRequest(score),
        ScoreResponse.class);
    }
}
```

Listing 12. Core/src/main/java/dk/sdu/mmmi/cbse/main/ScoreClient.java

The server side is a conventional Spring Boot REST controller [7] holding the score in an `AtomicInteger`, exposing GET and POST on `/api/score` and a reset endpoint. It carries no module-info: it is deployed on the class path as an ordinary executable jar, which reinforces that the boundary between the two processes is the HTTP contract and nothing else.

6 Test

Validation was carried out at three levels: unit tests of individual components, an integration run of the complete deployment in its real configuration, and a controlled experiment demonstrating dynamic update. All evidence in this section is reproduced from actual runs of the built application.

6.1 Unit testing strategy

The exercise asks for one unit test [12]. Twenty-one were written across five test classes, covering both examples the exercise names, moving the player ship and collision detection, and both testing styles its reading material describes. NFR5 is what makes this possible: because every component's logic is reachable through plain objects, no test requires a JavaFX stage.

Test class	Module	Tests	What it establishes
CollisionDetectorTest	Collision	6	Pythagorean distance including the 3-4-5 case, multi-hit destruction, ship destroyed by an asteroid without scoring, and removal of both sides of a collision
PlayerStateTest	Common	5	Lives decrement to game over, hits ignored during respawn invulnerability, one-shot hit consumption, and reset
EntityTest	Common	4	Default single-hit destruction, survival until max health is depleted, damage floored at zero, and refill on setMaxHealth
PlayerControlSystemTest	Player	3	Rotation on arrow keys, thrust followed by friction, and wrapping at the screen edge
AsteroidSizeTest	CommonAsteroids	3	The split chain large to medium to small, and that the smallest size has no successor

Table 13. Unit test inventory. 21 tests across 5 classes and 4 modules.

The two styles are deliberately contrasted within one class. The state-based test builds a real World, fires three bullets one frame at a time, and asserts that the enemy survives the first two

and is destroyed by the third. The interaction-based test replaces `World` with a Mockito mock [10] and verifies that `removeEntity` was called exactly twice, once per side of the collision, which is a claim about the interaction rather than about the resulting state.

```
@Test
void enemyShipIsDestroyedByAnAsteroidWithoutScoringForThePlayer() {
    GameData gameData = new GameData();
    World world = new World();

    Entity enemy = entityAt(100, 100, 8, EntityCategory.ENEMY);
    enemy.setMaxHealth(3); // survives 3 bullets, but not one asteroid
    world.addEntity(enemy);
    world.addEntity(entityAt(100, 100, 14, EntityCategory.ASTEROID));

    detector.process(gameData, world);

    assertFalse(world.getEntities().contains(enemy), "enemy should be destroyed by the
asteroid");
    assertEquals(0, gameData.getScoreState().getScore(), "a rock kill is not the player's
kill");
}
```

*Listing 14. A state-based test of the contract in Section 5.5.
Collision/src/test/java/dk/sdu/mmmi/cbse/collisionsystem/CollisionDetectorTest.java*

6.2 Build and unit test results

The full reactor builds and all tests pass. Figure 13 and Figure 14 are captured from the terminal session that produced the artefacts used everywhere else in this section.

```
mvn clean install - BUILD SUCCESS

[INFO] Copying Enemy-1.0.1-SNAPSHOT.jar to C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\plugins\Enemy-1.0.1-SNAPSHOT.jar
[INFO] -----< dk.sdu.mmmi.cbse:Scoring >-----
[INFO] Building Scoring 1.0.1-SNAPSHOT [11/11]
[INFO] from Scoring\pom.xml
[INFO] -----[ jar ]-----
[INFO] --- clean:3.2.0:clean (default-clean) @ Scoring ---
[INFO] Deleting C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target
[INFO] --- resources:3.4.0:resources (default-resources) @ Scoring ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] --- compiler:3.10.1:compile (default-compile) @ Scoring ---
[INFO] Changes detected - recompiling the module!
[INFO] Compiling 2 source files to C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target\classes
[INFO] --- resources:3.4.0:testResources (default-testResources) @ Scoring ---
[INFO] skip non existing resourceDirectory C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\src\test\resources
[INFO] --- compiler:3.10.1:testCompile (default-testCompile) @ Scoring ---
[INFO] No sources to compile
[INFO] --- surefire:3.2.5:test (default-test) @ Scoring ---
[INFO] No tests to run.
[INFO] --- jar:3.5.0:jar (default-jar) @ Scoring ---
[INFO] Building jar: C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target\Scoring-1.0.1-SNAPSHOT.jar
[INFO] --- spring-boot:3.2.4:repackage (default) @ Scoring ---
[INFO] Replacing main artifact C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target\Scoring-1.0.1-SNAPSHOT.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target\Scoring-1.0.1-SNAPSHOT.jar.original
[INFO] --- install:3.1.4:install (default-install) @ Scoring ---
[INFO] Installing C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\pom.xml to C:\Users\moval\.m2\repository\dk\sdu\mmmi\cbse\Scoring\1.0.1-SNAPSHOT\Scoring-1.0.1-SNAPSHOT.pom
[INFO] Installing C:\Users\moval\Desktop\Game-Project\full\AsteroidsFX-master\Scoring\target\Scoring-1.0.1-SNAPSHOT.jar to C:\Users\moval\.m2\repository\dk\sdu\mmmi\cbse\Scoring\1.0.1-SNAPSHOT\Scoring-1.0.1-SNAPSHOT.jar
[INFO] -----
[INFO] Reactor Summary for AsteroidsFX 1.0.1-SNAPSHOT:
[INFO] AsteroidsFX ..... SUCCESS [ 1.766 s]
[INFO] Common ..... SUCCESS [ 40.043 s]
[INFO] CommonBullet ..... SUCCESS [ 4.135 s]
[INFO] CommonAsteroids ..... SUCCESS [ 12.556 s]
[INFO] Core ..... SUCCESS [ 13.408 s]
[INFO] Player ..... SUCCESS [ 15.154 s]
[INFO] Bullet ..... SUCCESS [ 4.518 s]
[INFO] Asteroids ..... SUCCESS [ 12.571 s]
[INFO] Collision ..... SUCCESS [01:33 min]
[INFO] Enemy ..... SUCCESS [ 13.134 s]
[INFO] Scoring ..... SUCCESS [ 55.778 s]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 04:28 min
[INFO] Finished at: 2026-08-08T21:44:39+08:00
[INFO] -----
```

Figure 12. mvn clean install over all eleven Maven modules, ending in BUILD SUCCESS and populating mods-mvn/ and plugins/.

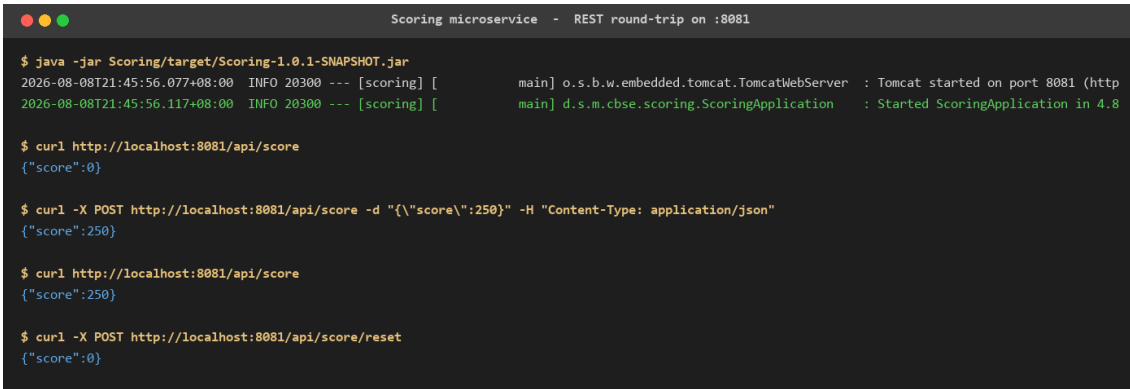
```
mvn test - 21 tests, 0 failures

[INFO] -----< dk.sdu.mmmi.cbse:Common >-----
[INFO] -----[ jar ]-----
[INFO] T E S T S
[INFO] Running dk.sdu.mmmi.cbse.common.data.EntityTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.290 s -- in dk.sdu.mmmi.cbse.common.data.EntityTest
[INFO] Running dk.sdu.mmmi.cbse.common.data.PlayerStateTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.077 s -- in dk.sdu.mmmi.cbse.common.data.PlayerStateTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----< dk.sdu.mmmi.cbse:CommonBullet >-----
[INFO] -----[ jar ]-----
[INFO] -----< dk.sdu.mmmi.cbse:CommonAsteroids >-----
[INFO] -----[ jar ]-----
[INFO] T E S T S
[INFO] Running dk.sdu.mmmi.cbse.common.asteroids.AsteroidSizeTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.209 s -- in dk.sdu.mmmi.cbse.common.asteroids.AsteroidSizeTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----< dk.sdu.cbse.asteroids:Core >-----
[INFO] -----[ jar ]-----
[INFO] -----< dk.sdu.mmmi.cbse:Player >-----
[INFO] -----[ jar ]-----
[INFO] T E S T S
[INFO] Running dk.sdu.mmmi.cbse.playersystem.PlayerControlSystemTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.261 s -- in dk.sdu.mmmi.cbse.playersystem.PlayerControlSystemTest
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----< dk.sdu.mmmi.cbse:Bullet >-----
[INFO] -----[ jar ]-----
[INFO] -----< dk.sdu.mmmi.cbse:Asteroids >-----
[INFO] -----[ jar ]-----
[INFO] -----< dk.sdu.mmmi.cbse:Collision >-----
[INFO] -----[ jar ]-----
[INFO] T E S T S
[INFO] Running dk.sdu.mmmi.cbse.collisionsystem.CollisionDetectorTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.614 s -- in dk.sdu.mmmi.cbse.collisionsystem.CollisionDetectorTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----< dk.sdu.mmmi.cbse:Enemy >-----
[INFO] -----[ jar ]-----
[INFO] -----< dk.sdu.mmmi.cbse:Scoring >-----
[INFO] -----[ jar ]-----
[INFO] Reactor Summary for AsteroidsFX 1.0.1-SNAPSHOT:
[INFO] BUILD SUCCESS
```

Figure 13. mvn test. All 21 tests pass across the four modules that carry tests.

6.3 Integration in a real deployment

Integration was validated by running the system exactly as it is meant to be deployed: two processes, the game launched from the module path and the scoring service as a standalone jar. Figure 15 shows the scoring service starting on port 8081 and answering a full round trip: read the score, set it, read it back, reset it.



```
Scoring microservice - REST round-trip on :8081

$ java -jar Scoring/target/Scoring-1.0.1-SNAPSHOT.jar
2026-08-08T21:45:56.077+08:00 INFO 20300 --- [scoring] [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8081 (http
2026-08-08T21:45:56.117+08:00 INFO 20300 --- [scoring] [main] d.s.m.cbse.scoring.ScoringApplication : Started ScoringApplication in 4.8

$ curl http://localhost:8081/api/score
{"score":0}

$ curl -X POST http://localhost:8081/api/score -d '{"score":250}' -H "Content-Type: application/json"
{"score":250}

$ curl http://localhost:8081/api/score
{"score":250}

$ curl -X POST http://localhost:8081/api/score/reset
{"score":0}
```

Figure 14. The scoring microservice starting and answering GET, POST and reset over HTTP.

Figure 16 shows the application at its start menu, and Figure 17 during play with the components cooperating: the Player component's ship, the Enemy component's saucer, an asteroid from the Asteroids component, and bullets from the Bullet component, with the Collision component arbitrating between them. Every entity visible in that frame was contributed by a module that Core does not name.



Figure 15. The application at start-up, in the `START_MENU` state, rendered by the Core component.

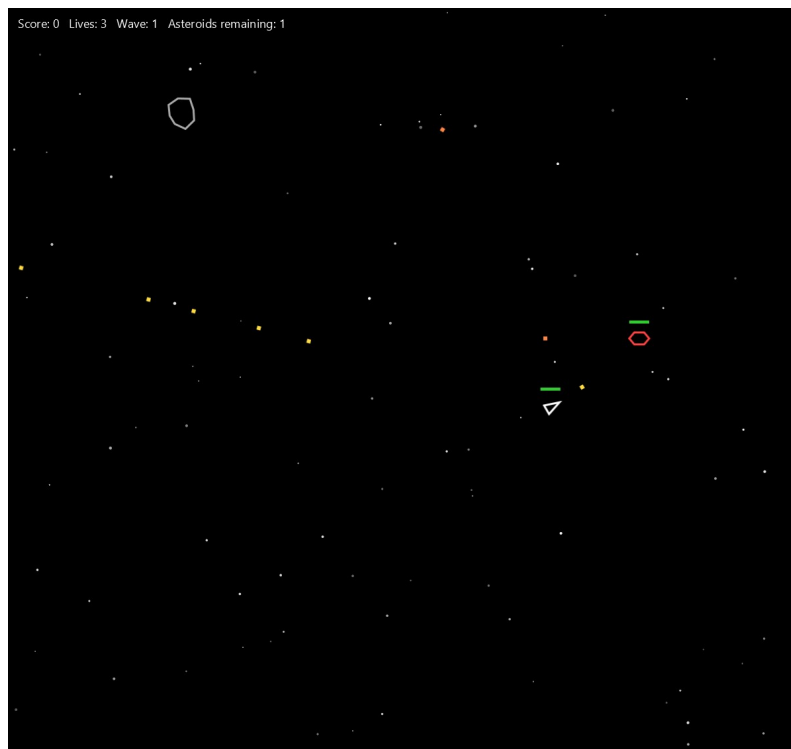


Figure 16. Gameplay with all five components installed. Player ship and enemy saucer both carry health bars; player bullets are yellow and enemy bullets orange.

6.4 Experimental validation of dynamic update

This is the experiment that addresses NFR1 and the deliverable required by the course information sheet. It was performed twice, at two different levels, because the two demonstrate different claims.

Experiment 1: uninstallation while the process runs.

With the game running, the Weapon component was uninstalled by pressing key 3, the fire key was pressed repeatedly, and the component was then reinstalled and the fire key pressed again. Figure 18 shows the three states. In the first, bullets are in flight. In the second, no bullet exists anywhere on the field despite repeated firing, because `BulletSPI.createBullet` is returning an empty `Optional`. In the third, firing works again. The player ship continues to fly throughout, and no other component is disturbed.

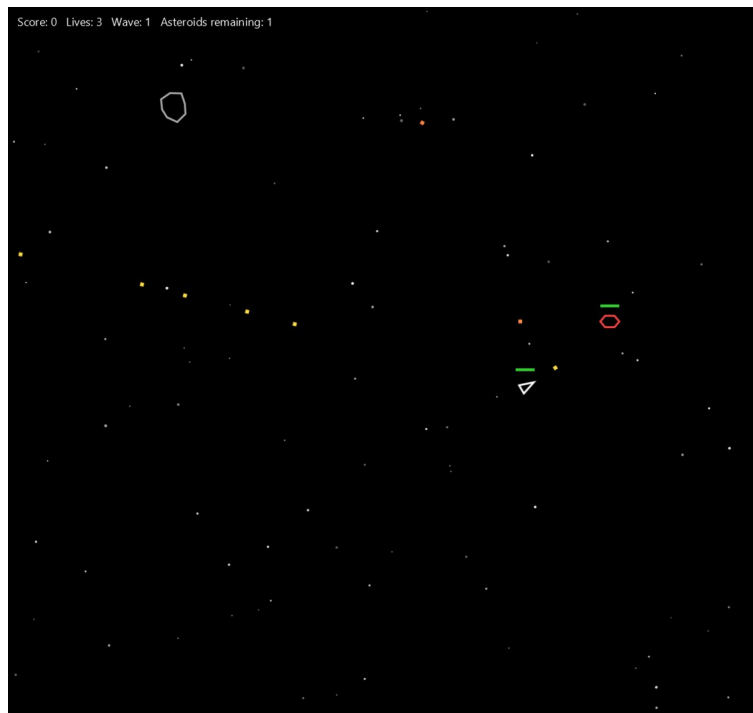


Figure 17. Weapon component installed: bullets in flight.

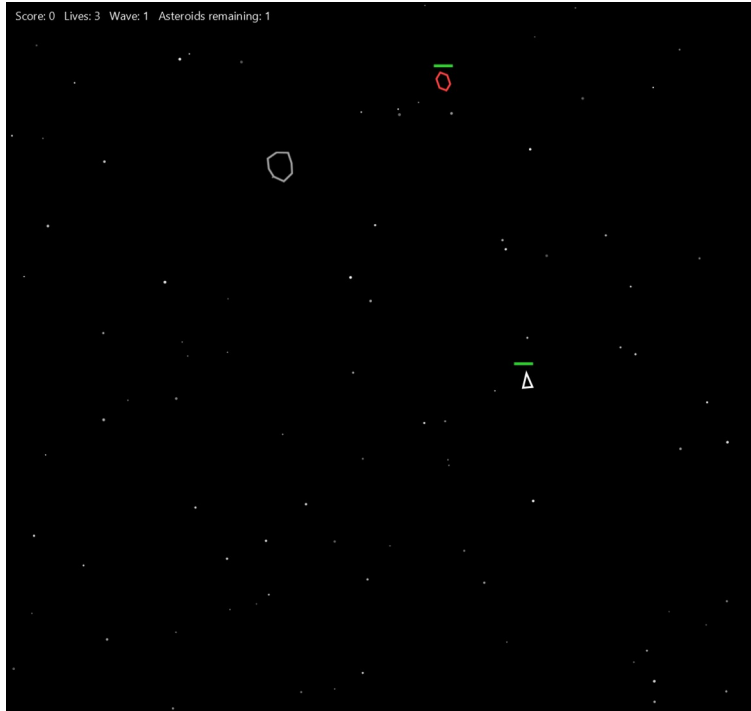


Figure 18. Weapon component uninstalled at runtime with key 3. The fire key was pressed repeatedly and no bullet exists on the field; the rest of the game continues normally.

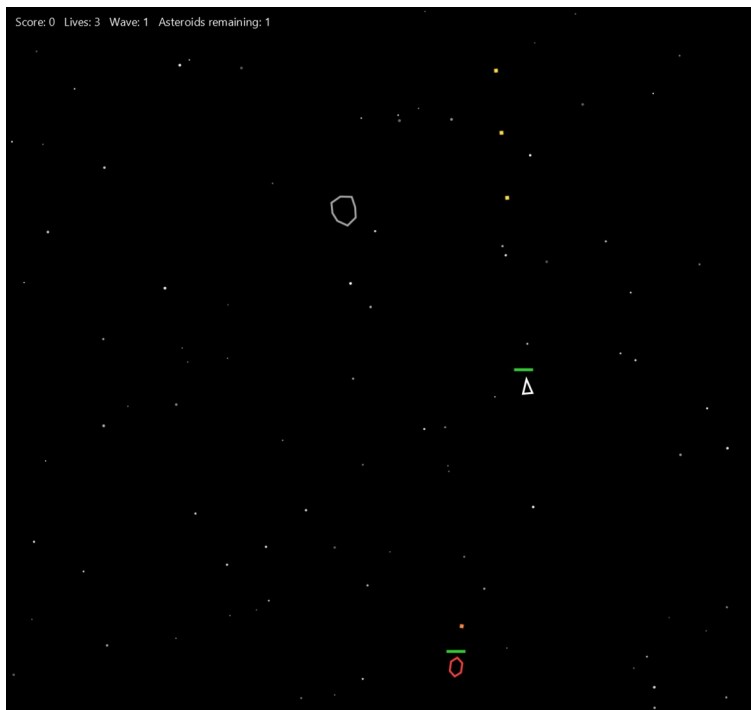
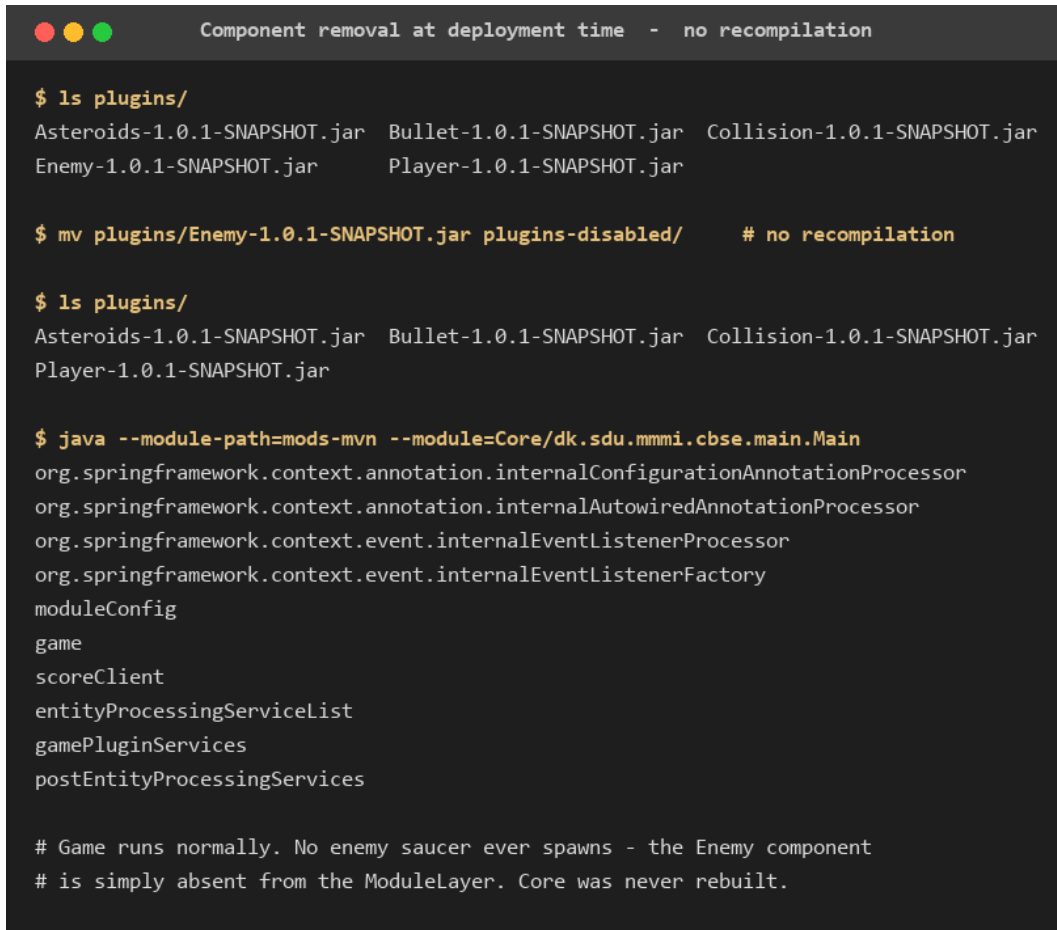


Figure 19. Weapon component reinstalled with key 3. Firing resumes immediately, with no recompilation and no restart of the JVM.

Experiment 2: removal from the deployment.

The first experiment shows a component being switched off inside a running process, but the instances were still discovered at start-up. The stronger claim is that a component need not be present at all. To test this, the Enemy component's jar was moved out of the plugins directory and the application was launched again. No source file was touched and no module was rebuilt.

A terminal window titled "Component removal at deployment time - no recompilation" with standard macOS window controls (red, yellow, green buttons). The terminal shows a sequence of commands and their outputs. First, a directory listing of 'plugins/' shows five JAR files: Asteroids-1.0.1-SNAPSHOT.jar, Bullet-1.0.1-SNAPSHOT.jar, Collision-1.0.1-SNAPSHOT.jar, Enemy-1.0.1-SNAPSHOT.jar, and Player-1.0.1-SNAPSHOT.jar. Then, a command moves 'Enemy-1.0.1-SNAPSHOT.jar' to 'plugins-disabled/'. A second directory listing of 'plugins/' shows only four JAR files, with 'Enemy-1.0.1-SNAPSHOT.jar' missing. Finally, a Java command is executed, which runs successfully and prints a list of module names. The list includes Spring Framework modules, 'moduleConfig', 'game', 'scoreClient', 'entityProcessingServiceList', 'gamePluginServices', and 'postEntityProcessingServices'. A comment at the bottom states: "# Game runs normally. No enemy saucer ever spawns - the Enemy component # is simply absent from the ModuleLayer. Core was never rebuilt."

```
Component removal at deployment time - no recompilation

$ ls plugins/
Asteroids-1.0.1-SNAPSHOT.jar  Bullet-1.0.1-SNAPSHOT.jar  Collision-1.0.1-SNAPSHOT.jar
Enemy-1.0.1-SNAPSHOT.jar      Player-1.0.1-SNAPSHOT.jar

$ mv plugins/Enemy-1.0.1-SNAPSHOT.jar plugins-disabled/ # no recompilation

$ ls plugins/
Asteroids-1.0.1-SNAPSHOT.jar  Bullet-1.0.1-SNAPSHOT.jar  Collision-1.0.1-SNAPSHOT.jar
Player-1.0.1-SNAPSHOT.jar

$ java --module-path=mods-mvn --module=Core/dk.sdu.mmmi.cbse.main.Main
org.springframework.context.annotation.internalConfigurationAnnotationProcessor
org.springframework.context.annotation.internalAutowiredAnnotationProcessor
org.springframework.context.event.internalEventListenerProcessor
org.springframework.context.event.internalEventListenerFactory
moduleConfig
game
scoreClient
entityProcessingServiceList
gamePluginServices
postEntityProcessingServices

# Game runs normally. No enemy saucer ever spawns - the Enemy component
# is simply absent from the ModuleLayer. Core was never rebuilt.
```

Figure 20. The Enemy component removed from the deployment by moving its jar out of plugins/. Core is not rebuilt.

The application started normally and played correctly with four components instead of five. No enemy saucer ever appeared, no enemy bullets were fired, and nothing failed: the module layer simply resolved a smaller set of modules, and the lists Spring injected were shorter by one entry.

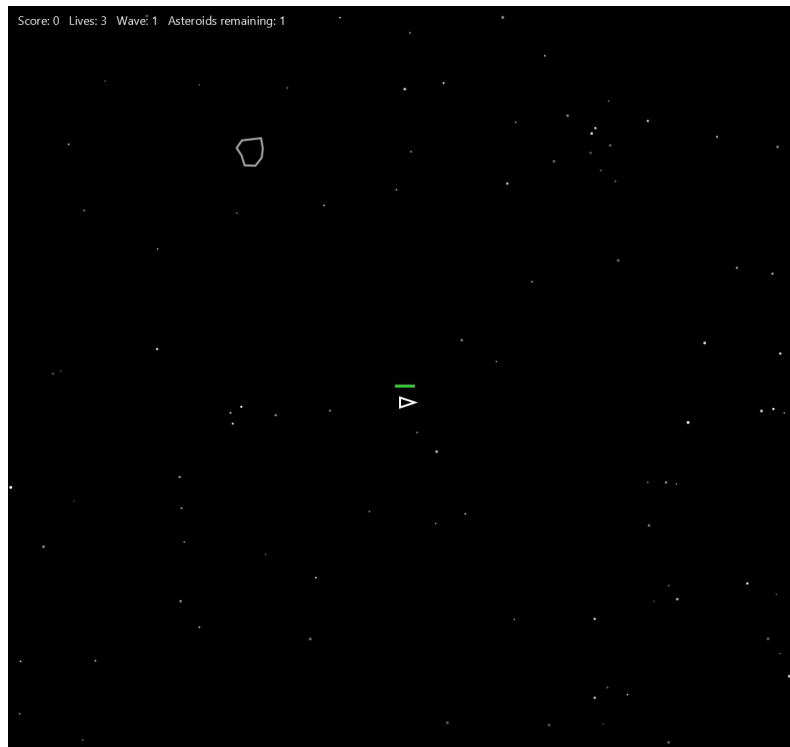


Figure 21. The game running with the Enemy component absent from the deployment. Player, Asteroids, Bullet and Collision continue to function; no saucer spawns.

The jar was then moved back and the enemy returned on the next launch. Taken together the two experiments establish NFR1 at both levels the exercise cares about: a component can be disabled in a live process, and a component can be absent from the deployment entirely, in both cases without recompiling the source code.

6.5 Summary of results

Requirement	Method of validation	Result
FR1 to FR11	Unit tests plus observation of the running game (Figures 16, 17)	Met
FR12	HTTP round trip against the running service (Figure 15)	Met
NFR1	Experiments 1 and 2 (Figures 18 to 22)	Met at both runtime and deployment level
NFR2	Inspection of Core/module-info.java; no gameplay module is named	Met
NFR3	Inspection of the five plugin descriptors; none exports a package	Met
NFR4	Resolution occurs in the ServiceLocator constructor and fails there if a module is missing	Met

NFR5	21 unit tests run with no JavaFX stage (Figure 14)	Met
NFR6	Game observed running normally with the scoring service stopped	Met

Table 15. Validation summary.

7 Discussion

7.1 How well the design met the requirements

The decisive requirement was NFR1, and the two experiments in Section 6.4 satisfy it directly. What is worth examining is why they succeed, because the outcome rests on a structural property rather than on a feature. Core cannot break when a component is removed because Core was never able to refer to that component in the first place. The compiler enforces this through the module descriptor, so the guarantee is not a matter of discipline that a later change might erode.

The same reasoning explains why the weapon experiment is the most informative of the three. Player and Enemy genuinely depend on a weapon to function as intended, and yet neither fails when it disappears, because the dependency is expressed as an Optional returned by a service rather than as a field of a known type. The contract anticipates absence, which is what distinguishes a component boundary from a mere package boundary.

7.2 Trade-offs accepted

Indirection has a cost, and it would be dishonest to present the architecture as free. Three costs are visible in the code.

- Traceability. Reading the source does not reveal which components will run; that is determined by a directory at start-up. The behaviour is discoverable only by running the application or listing plugins/, which is a real loss of static comprehensibility.
- Service lookup cost. Player and Enemy resolve BulletSPI through the module layer on each shot rather than caching a provider. At sixty frames per second with a firing cooldown this is not measurable, but it is a deliberate simplicity-over-efficiency choice rather than an oversight.
- Coordination through shared state. Collision cannot call into the Player component, so a hit is communicated by setting a flag in PlayerState that the Player component consumes on the next frame. This keeps the two components independent, but it moves part of the protocol into shared mutable state, where it is less explicit than a method call would be.

7.3 Limitations

The most significant limitation is that `ServiceLocator` constructs its module layer exactly once. A jar added to plugins/ after start-up is not seen, so Experiment 2 requires a relaunch even though it requires no recompilation. Supporting genuine hot deployment would mean building a new layer and migrating live entities between loaders, which is a substantially harder problem and well beyond what the exercise asks for. The boundary is therefore deliberate, and it is stated here rather than left for a reader to discover.

A second limitation is that the toggling mechanism groups service instances by the package name of their implementing class. This is a pragmatic mapping local to Core and it does not create a compile-time dependency, but it does encode a naming convention that a new component would have to follow to become toggleable. A more principled solution would carry the component name in the service contract itself.

Finally, the scoring service holds its state in memory. This satisfies the exercise, which asks for a microservice rather than for durable storage, but it means the externalised score is authoritative only for the lifetime of that process.

7.4 Reflection on the development process

Two defects found during development shaped the final design more than any diagram did. The first was the frozen-game symptom described in Section 4.6, which established that a call across a process boundary must be positioned so that its failure cannot abort work that has nothing to do with it. The second was found while auditing the implementation against the exercise text: enemy ships passed through asteroids untouched, because the collision table listed the player against asteroids but not the enemy. The exercise says ships, in the plural. The fix was small, but the lesson is that a requirement expressed in one word of prose can be missed entirely by an implementation that otherwise looks complete, which is precisely the argument for writing explicit contracts and tests against them.

8 Conclusion

This project set out to build an Asteroids game whose gameplay subsystems are genuine components, and to demonstrate that one of them can be removed and restored without recompiling the source code. Both objectives were met.

The resulting system is assembled from five gameplay components that the composition root cannot name. Their contracts are declared in shared API modules as JPMS services with explicit pre- and post-conditions, they are discovered at runtime by ServiceLoader from a plugins directory, they are resolved into their own ModuleLayer so that their resolution is isolated from the boot layer, and they are injected into the game loop by the Spring container. Score keeping was moved out of the process altogether, behind an HTTP contract, with the failure of that service contained so that it cannot affect the frame rate.

Validation was carried out on three levels. Twenty-one unit tests exercise the component contracts in isolation, with no JavaFX stage required. The complete two-process deployment was run and a score round trip confirmed end to end. Most importantly, two experiments established dynamic update at the two levels that matter: a component was uninstalled and reinstalled inside a live process, and a component was removed from the deployment entirely, in both cases with no recompilation of any source file.

The wider result is the one the course is aimed at. Each exercise replaced a direct dependency with a discovered one, and the cumulative effect is a composition root whose module descriptor names no gameplay module at all. That absence is not a stylistic preference; it is the property that makes the parts of this system replaceable, and it is what the experiments in Section 6.4 ultimately measure.

9 References

- [1] S. Mak and P. Bakker, Java 9 Modularity: Patterns and Practices for Developing Maintainable Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.
- [2] M. Reinhold, "JSR 376: Java Platform Module System," Java Community Process, Sep. 2017. [Online]. Available: <https://openjdk.org/projects/jigsaw/spec/>

- [3] Oracle Corporation, "Class ServiceLoader," Java Platform, Standard Edition API Specification, 2023. [Online]. Available: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/ServiceLoader.html>
- [4] Oracle Corporation, "Creating Extensible Applications," The Java Tutorials, 2023. [Online]. Available: <https://docs.oracle.com/javase/tutorial/ext/basics/spi.html>
- [5] C. Szyperski, Component Software: Beyond Object-Oriented Programming, 2nd ed. Boston, MA, USA: Addison-Wesley, 2002.
- [6] VMware, Inc., "Spring Framework Reference Documentation, version 6.1," 2024. [Online]. Available: <https://docs.spring.io/spring-framework/reference/>
- [7] VMware, Inc., "Spring Boot Reference Documentation, version 3.2," 2024. [Online]. Available: <https://docs.spring.io/spring-boot/docs/current/reference/html/>
- [8] OpenJFX Project, "JavaFX 21 API Documentation," 2023. [Online]. Available: <https://openjfx.io/javadoc/21/>
- [9] JUnit Team, "JUnit 5 User Guide," 2024. [Online]. Available: <https://junit.org/junit5/docs/current/user-guide/>
- [10] Mockito Contributors, "Mockito Framework Documentation," 2024. [Online]. Available: <https://site.mockito.org/>
- [11] R. C. Martin, "The Dependency Inversion Principle," C++ Report, vol. 8, no. 6, pp. 61-66, 1996.
- [12] Course teaching staff, "Component Based Systems: Laboratory Exercises (labs.pdf)," Maersk McKinney Moller Institute, University of Southern Denmark, 2026.
- [13] C. Larman, Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2004.
- [14] N. Llopis, "Data-Oriented Design (or Why You Might Be Shooting Yourself in the Foot with OOP)," Game Developer Magazine, pp. 11-13, Sep. 2009.
- [15] OSGi Alliance, "Listeners Considered Harmful: The Whiteboard Pattern," Technical Whitepaper revision 2.0, Aug. 2004.
- [16] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994.

- [17] J. Lewis and M. Fowler, "Microservices: a definition of this new architectural term," martinowler.com, Mar. 2014. [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [18] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D. dissertation, Univ. California, Irvine, CA, USA, 2000.

Appendix A Repository map

Path	Contents
Core/	Composition root: Main, Game, ModuleConfig, ComponentRegistry, HUDRenderer, ScoreClient
Common/	Entity, World, GameData and the state objects; the three service interfaces; ServiceLocator
CommonBullet/	Bullet entity and the BulletSPI contract
CommonAsteroids/	Asteroid entity, AsteroidSize and the IAsteroidSplitter contract
Player/	Player component: PlayerPlugin and PlayerControlSystem
Enemy/	Enemy component: EnemyPlugin and EnemyControlSystem
Bullet/	Weapon component: BulletPlugin and BulletControlSystem, provider of BulletSPI
Asteroids/	Asteroids component: plugin, processor and splitter implementation
Collision/	Collision component: CollisionDetector
Scoring/	Spring Boot scoring microservice, deployed as a standalone jar
mods-mvn/	Build output: boot layer module path
plugins/	Build output: component jars discovered at start-up
docs/	Architecture notes, microservice notes, the split-package demonstration, and this report
docs/report-figures/	Mermaid sources for every diagram and the captured screenshots used above

Table 16. Repository layout.

Appendix B Reproducing the results

Every figure and measurement in Section 6 can be reproduced from a clean checkout with the following commands, run from the repository root. A JDK of version 17 or newer is required.

```
git clone https://github.com/Fard121/asteroid_game.git
cd asteroid_game

mvn clean install                                # builds 11 modules, runs 21 tests,
                                                # populates mods-mvn/ and plugins/

java -jar Scoring/target/Scoring-1.0.1-SNAPSHOT.jar # terminal 2: scoring service, port
8081

mvn exec:exec --non-recursive                    # terminal 1: launches the game

curl http://localhost:8081/api/score              # terminal 3: score matches the in-game HUD
```

Listing 17. Full reproduction sequence.

To reproduce Experiment 2 of Section 6.4, move `plugins/Enemy-1.0.1-SNAPSHOT.jar` out of the directory and launch the game again. No rebuild is required, and the enemy saucer will not appear. Moving the jar back restores it on the following launch.

The `--non-recursive` flag matters: it runs the `exec` plugin once for the root project only. Without it, Maven repeats the launch for every child module.